```
!pip install pandas numpy scikit-learn matplotlib seaborn
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->
```

```
import pandas as pd
import numpy as np

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data"

columns = [
    "Sex", "Length", "Diameter", "Height", "Whole_weight",
    "Shucked_weight", "Viscera_weight", "Shell_weight", "Rings"
]

df = pd.read_csv(url, names=columns)
df.head()
```

	Sex	Length	Diameter	Height	Whole_weight	Shucked_weight	Viscera_weight	Shell_weight	Rings
0	M	0.455	0.365	0.095	0.5140	0.2245	0.1010	0.150	15
1	M	0.350	0.265	0.090	0.2255	0.0995	0.0485	0.070	7
2	F	0.530	0.420	0.135	0.6770	0.2565	0.1415	0.210	9
3	M	0.440	0.365	0.125	0.5160	0.2155	0.1140	0.155	10
4	I	0.330	0.255	0.080	0.2050	0.0895	0.0395	0.055	7

後續步驟： 使用 df 生成程式碼 New interactive sheet

Q1. Simple Linear Regression: Diameter ~ Length

```
from sklearn.linear_model import LinearRegression

X = df[["Length"]]
y = df["Diameter"]

model = LinearRegression()
model.fit(X, y)

a = model.coef_[0]
b = model.intercept_

corr = np.corrcoef(df["Length"], df["Diameter"])[0,1]

print("Model: Diameter = a*Length + b")
print("a =", a)
print("b =", b)
print("Correlation =", corr)
```

```
Model: Diameter = a*Length + b
a = 0.8154606917560954
b = -0.01941370551997662
Correlation = 0.9868115846024929
```

Q2. Multilinear Regression: Whole Weight ~ Shucked + Viscera + Shell

Multilinear Regression

```
X = df[["Shucked_weight", "Viscera_weight", "Shell_weight"]]
y = df["Whole_weight"]

model = LinearRegression()
model.fit(X, y)

print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Coefficients: [0.93656021 1.11164951 1.25296164]
Intercept: -0.00782983143729954
```

Check

(a) Calculate the sum of the three.

```
df["sum_parts"] = (
    df["Shucked_weight"] +
    df["Viscera_weight"] +
    df["Shell_weight"]
)

df[["Whole_weight", "sum_parts"]].head()
```

	Whole_weight	sum_parts
0	0.5140	0.4755
1	0.2255	0.2180
2	0.6770	0.6080
3	0.5160	0.4845
4	0.2050	0.1840

(b) Look at the correlation coefficient. P.S The closer it is to 1, the stronger the correlation.

```
corr = np.corrcoef(df["Whole_weight"], df["sum_parts"])[0,1]
print("Correlation between Whole and (Shucked+Viscera+Shell):", corr)
```

```
Correlation between Whole and (Shucked+Viscera+Shell): 0.9950547929901482
```

(c) Look at the difference (Whole – sum_parts)

```
df["diff"] = df["Whole_weight"] - df["sum_parts"]
df["diff"].describe()
```

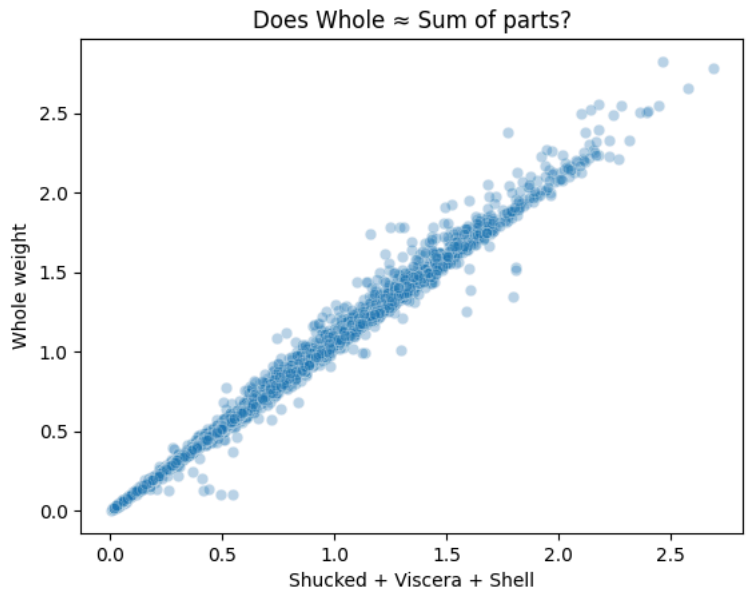
	diff
count	4177.000000
mean	0.049950
std	0.058072
min	-0.447500
25%	0.018000
50%	0.037000
75%	0.068000
max	0.608000

dtype: float64

(d) Scatter plot

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(x=df["sum_parts"], y=df["Whole_weight"], alpha=0.3)
plt.xlabel("Shucked + Viscera + Shell")
plt.ylabel("Whole weight")
plt.title("Does Whole ≈ Sum of parts?")
plt.show()
```

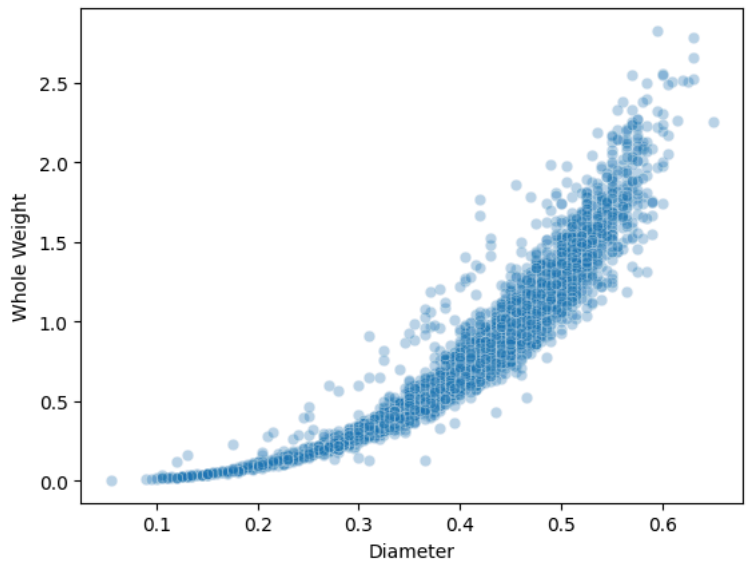


Q3. Non-linear models: Whole Weight vs Diameter

scatter plot

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(x=df["Diameter"], y=df["Whole_weight"], alpha=0.3)
plt.xlabel("Diameter")
plt.ylabel("Whole Weight")
plt.show()
```



(a) Linear: weight = a*d + b

```
X = df[["Diameter"]]
y = df["Whole_weight"]
```

```
model = LinearRegression().fit(X, y)

pred = model.predict(X)
corr = np.corrcoef(y, pred)[0,1]

print("Linear corr =", corr)
```

```
Linear corr = 0.9254521015071309
```

(b) Quadratic: weight = ad + bd² + c

```
df["d2"] = df["Diameter"]**2
X = df[["Diameter", "d2"]]
model = LinearRegression().fit(X, y)

pred = model.predict(X)
corr = np.corrcoef(y, pred)[0,1]

print("Quadratic corr =", corr)
```

```
Quadratic corr = 0.962684866053714
```

(c) Cubic no lower terms: weight = a*d³

```
df["d3"] = df["Diameter"]**3
X = df[["d3"]]
model = LinearRegression().fit(X, y)

pred = model.predict(X)
corr = np.corrcoef(y, pred)[0,1]

print("Cubic corr =", corr)
```

```
Cubic corr = 0.9630786738388938
```

(d) Exponential: log(weight) = a*d + b

```
df["logW"] = np.log(df["Whole_weight"])

X = df[["Diameter"]]
y_log = df["logW"]

model = LinearRegression().fit(X, y_log)

pred_log = model.predict(X)
corr = np.corrcoef(y_log, pred_log)[0,1]

print("Exponential corr =", corr)
```

```
Exponential corr = 0.9635120791439449
```

Q4. Logistic Regression — Infant (I) vs Adult (M/F)

Recoding

```
df["Infant"] = (df["Sex"] == "I").astype(int)
```

(a) Using length only

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def logistic_accuracy(features):
    X = df[features]
    y = df["Infant"]

    model = LogisticRegression(max_iter=1000)
    model.fit(X, y)
    pred = model.predict(X)
```

```
    return accuracy_score(y, pred)

print("Length only:", logistic_accuracy(["Length"]))
```

```
Length only: 0.7799856356236533
```

(b) Whole weight only

```
print("Whole weight only:", logistic_accuracy(["Whole_weight"]))
```

```
Whole weight only: 0.7955470433325353
```

(c) Rings only

```
print("Rings only:", logistic_accuracy(["Rings"]))
```

```
Rings only: 0.7881254488867608
```

(d) Length + whole weight + rings

```
print("Combined:", logistic_accuracy(["Length","Whole_weight","Rings"]))
```

```
Combined: 0.8139813263107494
```

Q5. Adult dataset logistic regression (Predict Sex)

Read Data

```
adult_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"

adult_cols = [
    "age","workclass","fnlwgt","education","education-num","marital-status",
    "occupation","relationship","race","sex","capital-gain","capital-loss",
    "hours-per-week","native-country","income"
]

adult = pd.read_csv(adult_url, names=adult_cols, na_values=" ?", skipinitialspace=True)
adult = adult.dropna()
```

One-hot encoding

```
adult_enc = pd.get_dummies(adult, drop_first=True)
```

Target

```
y = (adult["sex"] == "Male").astype(int)
X = adult_enc.drop(columns=["sex_Male"])
```

Build logistic regression claulate accuracy：

```
model = LogisticRegression(max_iter=500)
model.fit(X, y)
pred = model.predict(X)

accuracy = accuracy_score(y, pred)
accuracy
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed t
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
```

```
        https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
      n_iter_i = _check_optimize_result(
    0.808144713000215
```